P.Vishwa Nandan(192473015)

1. Selection of Java Data Types

Different Java data types are selected to store parking information.

1. **Slot Number – int:** Stores the parking slot number because it is a whole number.
2. **Vehicle Registration Number – String:** Stores vehicle numbers containing letters and digits.
3. **Entry Time – LocalDateTime:** Stores the date and time when a vehicle enters the parking facility.
4. **Parking Duration – long:** Stores the total parking duration in minutes.
5. **Parking Fee – double:** Stores parking charges, including decimal values.
6. **Floor Number – int:** Stores the floor number of the parking facility.

2. Representation of Floors and Parking Slots

A jagged two-dimensional array is selected to represent the parking floors and slots.

Array declaration:

String[][] slots;

Each row represents a parking floor, and each column represents a parking slot.

A jagged array is suitable because different floors can have different numbers of parking slots.

Example:

- Floor 0 contains 3 slots.
- Floor 1 contains 2 slots.
- Floor 2 contains 4 slots.

Java implementation:

```
int[] slotsPerFloor = {3, 2, 4};


String[][] slots = new String[slotsPerFloor.length][];


for (int f = 0; f < slotsPerFloor.length; f++) {
   slots[f] = new String[slotsPerFloor[f]];
}
```

An empty slot is represented using null, while an occupied slot stores the vehicle registration number.

A jagged array provides flexibility when the parking facility has different numbers of slots on different floors.

3. Type Conversion and Casting

Type conversion is required when calculating parking duration and fees.

For example, converting parking minutes into hours requires division by 60.

Integer division can produce incorrect results because it removes the decimal portion.

Example:

```
int minutes = 90;


int hours = minutes / 60;


System.out.println(hours);
```

Output:

1

The result is 1 instead of 1.5 because integer division removes the decimal portion.

If every started hour is charged fully, 90 minutes must be charged as 2 hours.

To avoid this problem, the following formula is used:

```
int minutes = 90;


int hours = (minutes + 59) / 60;


System.out.println(hours);
```

Output:

2

If the parking fee is calculated proportionally, floating-point division can be used:

```
double hours = minutes / 60.0;
```

This produces 1.5 hours without losing precision.

Therefore, the conversion method is selected according to the parking pricing rule.

4. Input Method

The Scanner class is selected to read input from the user.

It allows users to enter vehicle registration numbers, vehicle types and parking durations through the keyboard.

Example:

```
import java.util.Scanner;


class Main {
   public static void main(String[] args) {


      Scanner sc = new Scanner(System.in);


      System.out.println("Enter vehicle number:");
      String vehicleNo = sc.nextLine();


      System.out.println("Enter parking duration:");
      int minutes = sc.nextInt();


      System.out.println("Vehicle Number: " + vehicleNo);
      System.out.println("Parking Duration: " + minutes);


      sc.close();
   }
}
```

Scanner is preferred because the parking system requires interactive user input. Command-line arguments can also be used when input values are provided during program execution.

5. Alternative Concept and Complexity Analysis

Instead of using a jagged array, parking slots can be represented using a one-dimensional array or an ArrayList<String[]>.

A. One-Dimensional Array

A single array can store all parking slots.

For floors having an equal number of slots, the index can be calculated using:

index = floor × slotsPerFloor + slot

However, this formula does not directly work when floors have different numbers of slots.

For different slot capacities, prefix offsets can be maintained to calculate the correct index.

B. ArrayList<String[]>

An ArrayList can store separate String arrays for each parking floor.

Each floor can have a different number of slots. This representation is flexible and allows floors to be added dynamically.

Complexity Analysis

1. Jagged Array

- Accessing a known slot: O(1).
- Searching for an available slot: O(S).
- Space complexity: O(S + F).

2. One-Dimensional Array

- Accessing a known slot: O(1) with a suitable index calculation or prefix offsets.
- Searching for an available slot: O(S).
- Space complexity: O(S), or O(S + F) when prefix offsets are maintained.

3. ArrayList<String[]>

- Accessing a known floor and slot: O(1).
- Searching for an available slot: O(S).
- Space complexity: O(S + F), including collection overhead.

Here:

- F represents the number of floors.
- S represents the total number of parking slots.

6. Conclusion

A jagged two-dimensional array is selected because it supports different numbers of parking slots on different floors. Suitable Java data types and correct type conversion help maintain accurate parking information and fee calculations. Scanner is used to accept input from users.